```
# TEST 1 (8D-internal): does the relation table alone fix the order-30 cycle and its four channels,
# uniquely up to the table's own symmetries?  Input: the 240x240 integer relation table ONLY.
# (The table is produced once from the standard E8 element list, then every coordinate is deleted.)
import numpy as np, itertools, random
# ---- build the table, then discard coordinates ----
rt=[]
for i,j in itertools.combinations(range(8),2):
    for si in (2,-2):
        for sj in (2,-2):
            v=np.zeros(8,int); v[i]=si; v[j]=sj; rt.append(v)
for s in itertools.product([1,-1],repeat=8):
    if sum(x<0 for x in s)%2==0: rt.append(np.array(s))
G=(np.array(rt)@np.array(rt).T)//4; del rt          # integer relation table; coordinates gone
N=len(G);
print("table:",G.shape,"values",sorted(set(G.flatten())),"diagonal",set(np.diag(G)))
# ---- a basis from the table: 8 elements with nonsingular sub-table ----
B=[0]
for r in range(N):
    if np.linalg.matrix_rank(G[np.ix_(B+[r],B+[r])])>len(B): B.append(r)
    if len(B)==8: break
GB=G[np.ix_(B,B)].astype(float); GBi=np.linalg.inv(GB)
C=(GBi@G[B,:]).T                                     # coefficients of every element in the basis, from the table only
print("coefficients integral:",np.allclose(C,np.round(C)),"| table reproduced from coefficients:",np.allclose(C@GB@C.T,G))
Kf=lambda c: tuple(np.round(np.asarray(c)*840).astype(int))   # basis need not be a lattice basis: coefficients are rational
key={Kf(c):i for i,c in enumerate(C)}
print('distinct elements recovered from coefficients:',len(key))
def refl(a):                                         # reflection in element a, as a permutation of the 240: r -> r - G[a,r] a
    return np.array([key[Kf(C[r]-G[a,r]*C[a])] for r in range(N)])
def coxeter(seed):
    rng=np.random.RandomState(seed); w=rng.randn(8)
    h=C@w; pos=[r for r in range(N) if h[r]>0]       # positive system from a generic height (table-derived coefficients)
    P=set(pos); simple=[r for r in pos if not any(key.get(Kf(C[r]-C[a])) in P for a in pos if a!=r)]
    order=simple[:]; random.Random(seed).shuffle(order)
    perm=np.arange(N)
    for a in order: perm=refl(a)[perm]               # apply reflections in a random order
```

```
    return simple,perm
def order_of(p):
    q=p.copy(); k=1
    while not (q==np.arange(N)).all(): q=p[q]; k+=1
    return k
def channels(perm):
    M=np.array([C[perm[b]] for b in B]).T            # action on the basis, in basis
coefficients
    w,V=np.linalg.eig(M); m=np.round(np.angle(w)/(2*np.pi)*30).astype(int)%30
    # orthogonal projector onto each eigen-channel w.r.t. the table's own inner
product GB
    Pm={}
    for k in set(m):
        Vk=V[:,m==k]; Pm[k]=Vk@np.linalg.inv(Vk.conj().T@GB@Vk)@Vk.conj().T@GB
    return sorted(set(m)),Pm
def sector_sq(Pm,ms):
    P=sum(Pm[k] for k in ms); X=C@P.T
    return np.real(np.einsum('ri,ij,rj->r',X.conj(),GB,X))
res=[]
for seed in range(12):
    simple,perm=coxeter(seed)
    o=order_of(perm); orbits=len(set(min(perm_pow) for perm_pow in [[
(np.linalg.matrix_power(np.eye(1),0), )]]) ) if False else None
    seen=set(); orb=[]
    for r in range(N):
        if r in seen: continue
        cyc=[r]; x=perm[r]
        while x!=r: cyc.append(x); x=perm[x]
        seen|=set(cyc); orb.append(len(cyc))
    ms,Pm=channels(perm); lv=np.round(sector_sq(Pm,[1,29,11,19]),6)
    res.append((simple,perm))
    print(f"seed {seed:2d}: simple elements {len(simple)}, cycle order {o}, orbits
{len(orb)}x{set(orb)}, channel labels {ms}, "
          f"{{1,11}} levels {dict((float(k),int((lv==k).sum())) for k in
sorted(set(lv)))}")

# ---- uniqueness: is every cycle obtained a relabeling of the first by a symmetry of
the table? ----
# search for a permutation s of the 240 with G[s(a),s(b)] = G[a,b] for all a,b  and
s(c1(r)) = c2(s(r)) for all r
def orbits_of(p):
    seen=set(); out=[]
    for r in range(N):
        if r in seen: continue
        cyc=[r]; x=p[r]
        while x!=r: cyc.append(x); x=p[x]
        seen|=set(cyc); out.append(cyc)
    return out
def conjugator(p1,p2):
    O=orbits_of(p1)
```

```
    def place(k,s):
        if k==len(O): return s
        cyc=O[k]
        for x in range(N):
            if x in s.values(): continue
            t=dict(s); y=x; ok=True
            for r in cyc:
                if y in t.values(): ok=False; break
                t[r]=y; y=p2[y]
            if not ok: continue
            dom=list(t.keys()); img=[t[d] for d in dom]
            if (G[np.ix_(cyc,dom)]==G[np.ix_([t[r] for r in cyc],img)]).all():
                out=place(k+1,t)
                if out is not None: return out
        return None
    return place(0,{})
p1=res[0][1]; inner1=set(np.where(np.round(sector_sq(channels(p1)[1],[1,29,11,19]),6)
<1)[0])
print("\nrelabeling search against the first cycle:")
for seed in range(1,12):
    p2=res[seed][1]; s=conjugator(p1,p2)
    if s is None: print(f"  seed {seed}: NO table symmetry relates the two cycles");
continue
    S=np.array([s[r] for r in range(N)])
    tab=(G[np.ix_(S,S)]==G).all(); com=(S[p1]==p2[S]).all()
    inner2=set(np.where(np.round(sector_sq(channels(p2)[1],[1,29,11,19]),6)<1)[0])
    print(f"  seed {seed}: symmetry found | preserves whole table: {tab} | carries
cycle 1 onto cycle {seed}: {com} | carries inner level onto inner level:
{set(S[list(inner1)])==inner2}")
```